Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____С.В.Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №4
по дисциплине «Инженерно-техническая поддержка сопровождения интеллектуальных информационных систем»
«Контейнеризация компонентов ИИС и управление зависимостями (Docker)»
ЛР 09.02.08.03.004

Студент группы 1ИИС-41 Р.Н. Анненко
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Лабораторная работа №4

Контейнеризация компонентов ИИС и управление зависимостями (Docker)

Цели и задачи работы:
освоение принципов контейнеризации интеллектуальных информационных систем (ИИС) с использованием Docker; создание Dockerfile для приложений на языке Python с тяжёлыми зависимостями машинного обучения (NumPy, Scikit-learn); оптимизация размера Docker-образов методами multi-stage build и использования базовых образов slim; описание многоконтейнерного приложения с помощью docker-compose.yml (веб-интерфейс ИИС и база данных PostgreSQL); организация персистентного хранения состояния системы через volumes.

Задание к работе:
выполнить шаги 1 – 10 в соответствии с условиями. Дополнительное задание (п. 11, кэширование предсказаний в Redis) не выполнялось.

Ход работы

Шаг 1. Установка Docker и проверка версий

Я установил Docker Engine и плагин Docker Compose из официального репозитория Docker для Ubuntu: добавил GPG-ключ и репозиторий, затем установил пакеты docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin и docker-compose-plugin. Пользователя добавил в группу docker, чтобы команды выполнялись без sudo.

Работа выполнялась в виртуальной машине со следующими параметрами: 2 vCPU, 3,3 ГБ оперативной памяти, 2 ГБ swap, диск 23 ГБ (свободно 7,8 ГБ).

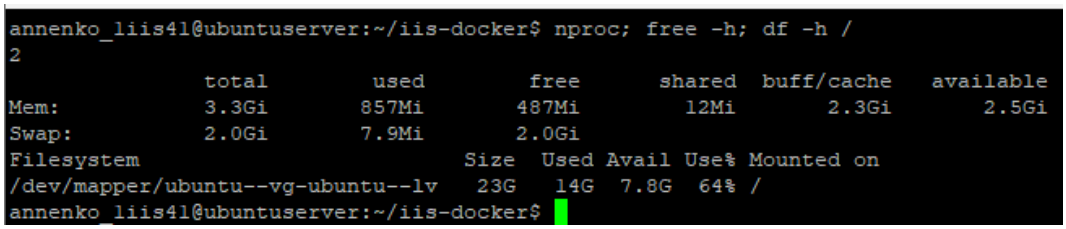


Рисунок 1 – Параметры виртуальной машины (nproc, free -h, df -h)

Проверил версии команд: Docker 29.9.2, Docker Compose v5.7.0. Тестовый контейнер hello-world запустился успешно.

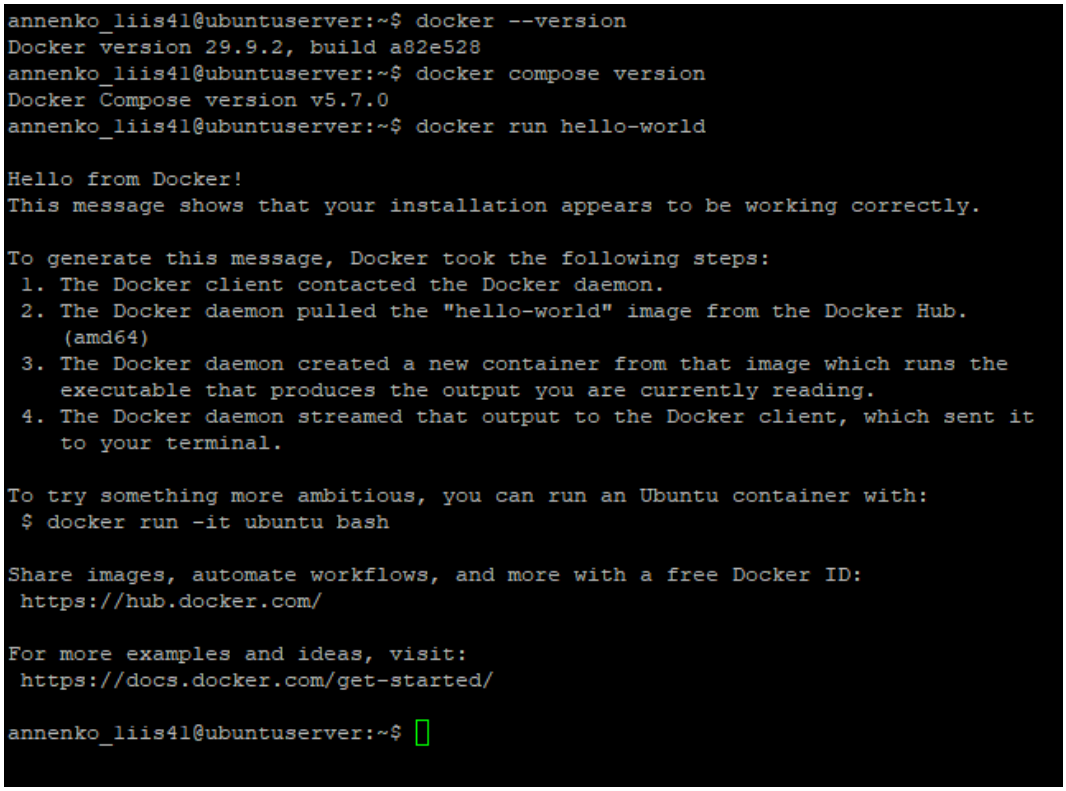


Рисунок 2 – Проверка версий Docker и запуск контейнера hello-world

Шаг 2. Создание структуры проекта

Я создал каталог iis-docker и в нём структуру проекта: каталог app с файлами main.py (HTTP-API на FastAPI), model.py (обучение и загрузка модели RandomForest на наборе Iris) и requirements.txt (зависимости Python), а также Dockerfile, Dockerfile.naive, docker-compose.yml, .dockerignore и каталог logs для логов.

Структура каталога проверена командой tree -a.

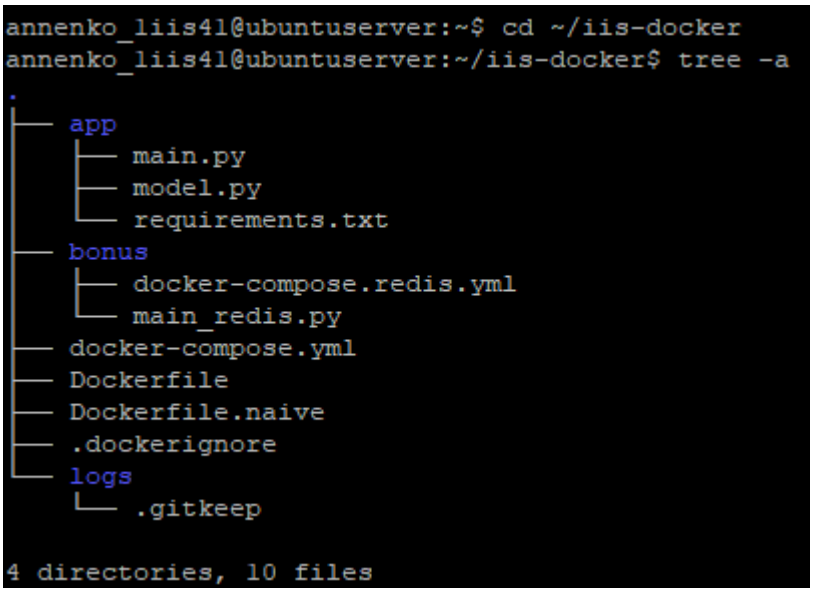


Рисунок 3 – Структура каталога проекта iis-docker

Шаг 3. Dockerfile с multi-stage сборкой

Dockerfile состоит из двух стадий. Стадия builder основана на образе python:3.11-slim: в ней устанавливаются системные пакеты, необходимые только для компиляции (build-essential, gcc), создаётся виртуальное окружение /opt/venv и устанавливаются зависимости из requirements.txt с флагом --no-cache-dir.

Стадия runtime также основана на python:3.11-slim и содержит только результат: из стадии builder копируется готовое виртуальное окружение (COPY --from=builder /opt/venv /opt/venv) и исходный код приложения. Компиляторы в итоговый образ не попадают.

Приложение запускается от непривилегированного пользователя appuser. Дополнительно в стадии runtime создаётся каталог /data и передаётся пользователю appuser: без этого модель не может быть записана в смонтированный volume, так как новый том принадлежит root, и joblib.dump завершается ошибкой Permission denied.

```
annenko_iis41@ubuntuserver:~/iis-docker$ cat Dockerfile
# ===== Стадия 1: сборка зависимостей =====
FROM python:3.11-slim AS builder

WORKDIR /build

# Системные пакеты, нужные только на этапе компиляции
# (в финальный образ не попадут)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc \
    && rm -rf /var/lib/apt/lists/*

# Копируем только список зависимостей — для кэширования слоя pip
COPY app/requirements.txt .

# Виртуальное окружение изолирует пакеты Python
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# --no-cache-dir уменьшает размер слоя (не сохраняется кэш pip)
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# ===== Стадия 2: исполняемый образ =====
FROM python:3.11-slim AS runtime

WORKDIR /app

# Копируем только venv из стадии builder (без компиляторов)
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Исходный код приложения
COPY app/ .

# Непривилегированный пользователь (рекомендация по безопасности).
# Каталог /data создаём заранее и отдаём appuser, иначе модель нельзя будет
# записать в смонтированный volume (Permission denied).
RUN useradd -m -u 1000 appuser \
    && chown -R appuser:appuser /app \
    && mkdir -p /data \
    && chown appuser:appuser /data
USER appuser

EXPOSE 8000

# Запуск ASGI-сервера uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Рисунок 4 – Содержимое файла Dockerfile

Шаг 4. Файл .dockerignore

В файле .dockerignore перечислено то, что не должно попадать в контекст сборки: __pycache__, *.pyc, .git, .env, *.md, .venv, logs и *.log. Это уменьшает объём контекста и исключает попадание в образ служебных файлов и секретов.

```
annenko_iis41@ubuntuserver:~/iis-docker$ cat .dockerignore
# Исключения из контекста сборки Docker
__pycache__
*.pyc
.git
.env
*.md
.venv
logs
*.log
annenko_iis41@ubuntuserver:~/iis-docker$
```

Рисунок 5 – Содержимое файла .dockerignore

Шаг 5. Описание docker-compose.yml

Файл описывает два сервиса. Сервис api собирается из локального Dockerfile, публикует порт 8000, монтирует именованный том model_data в каталог /data и bind-mount ./logs в /app/logs, получает строку подключения DATABASE_URL и зависит от сервиса db.

```
annenko_iis41@ubuntuserver:~/iis-docker$ cat docker-compose.yml
# В новых версиях Docker Compose атрибут version считается устаревшим
# и выводит безвредное предупреждение — оставлен для соответствия методичке.
version: "3.9"

services:
  api:
    build: .                        # сборка из Dockerfile в текущем каталоге
    ports:
      - "8000:8000"                 # публикация порта API на хосте
    volumes:
      - model_data:/data            # named volume для файла модели
      - ./logs:/app/logs            # bind-mount для логов (необязательно)
    environment:
      - DATABASE_URL=postgresql://iis:iispass@db:5432/iisdb
    depends_on:
      - db                          # порядок запуска (не ждёт готовности БД)
    restart: unless-stopped

  db:
    image: postgres:16-alpine       # лёгкий официальный образ PostgreSQL
    environment:
      POSTGRES_USER: iis
      POSTGRES_PASSWORD: iispass
      POSTGRES_DB: iisdb
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    restart: unless-stopped

volumes:
  model_data:                       # данные модели ML
  postgres_data:                    # данные СУБД
```

Рисунок 6 – Содержимое файла docker-compose.yml

Сервис db использует официальный образ postgres:16-alpine, хранит данные в именованном томе postgres_data и публикует порт 5432. Оба тома объявлены

в разделе volumes. Корректность файла проверена командой docker compose config.

```
annenko_liis41@ubuntuserver:~/iis-docker$ docker compose config
WARN[0000] /home/annenko_liis41/iis-docker/docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion
name: iis-docker
services:
  api:
    restart: unless-stopped
    environment:
      DATABASE_URL: postgresql://iis:iispass@db:5432/iisdb
    networks:
      default: null
    volumes:
      - type: volume
        source: model_data
        target: /data
        volume: {}
      - type: bind
        source: /home/annenko_liis41/iis-docker/logs
        target: /app/logs
        bind: {}
    build:
      context: /home/annenko_liis41/iis-docker
      dockerfile: Dockerfile
    depends_on:
      db:
        condition: service_started
        required: true
    ports:
      - mode: ingress
        target: 8000
        published: "8000"
        protocol: tcp
  db:
    restart: unless-stopped
    environment:
      POSTGRES_DB: iisdb
      POSTGRES_PASSWORD: iispass
      POSTGRES_USER: iis
    image: postgres:16-alpine
    networks:
      default: null
    volumes:
      - type: volume
        source: postgres_data
        target: /var/lib/postgresql/data
        volume: {}
    ports:
      - mode: ingress
        target: 5432
        published: "5432"
        protocol: tcp
networks:
  default:
    name: iis-docker_default
volumes:
  model_data:
    name: iis-docker_model_data
  postgres_data:
    name: iis-docker_postgres_data
annenko_liis41@ubuntuserver:~/iis-docker$
```

Рисунок 7 – Результат проверки docker compose config

Шаг 6. Сборка образов и сравнение размеров

Для сравнения я собрал два образа. Первый — по файлу Dockerfile.naive: одна стадия на полном образе python:3.11, с установленными компиляторами и запуском от root. Его размер составил 2,41 ГБ.

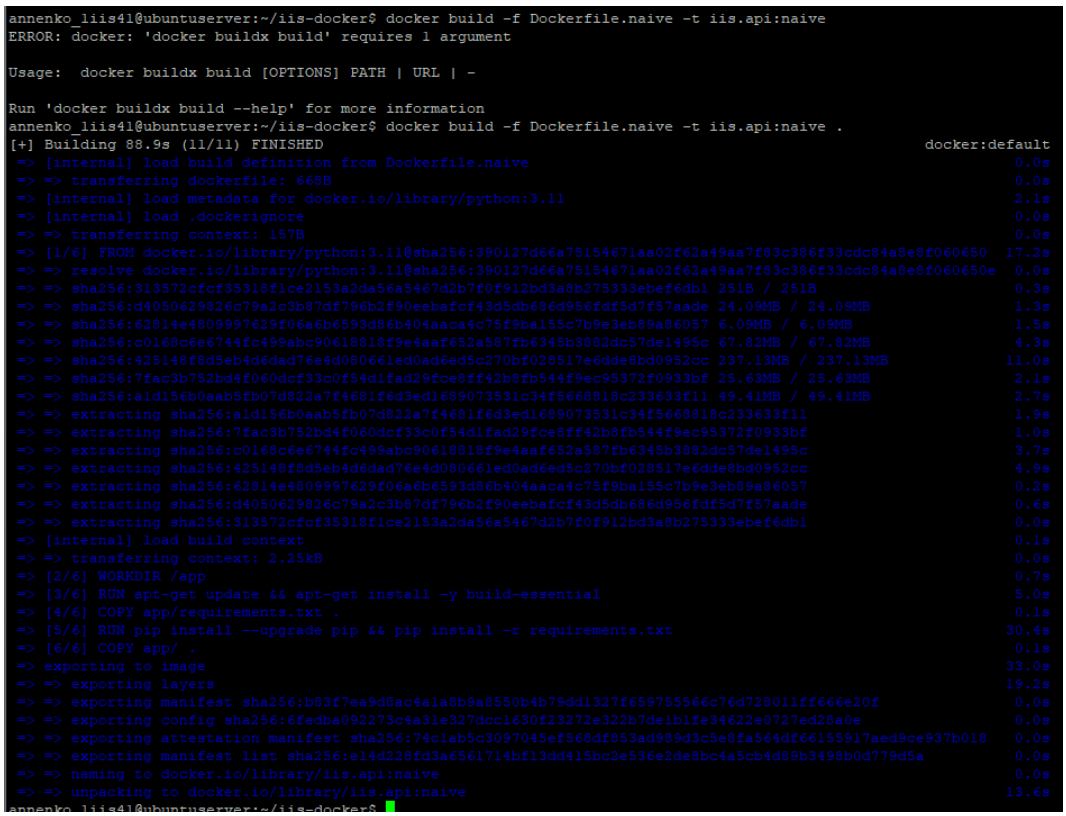


Рисунок 8 – Сборка образа iis-api:naive по одностадийному Dockerfile

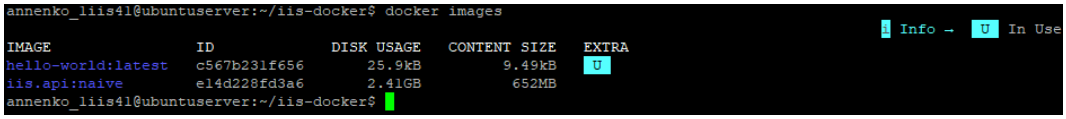


Рисунок 9 – Размер образа iis-api:naive (2,41 ГБ)

Второй образ собран по основному Dockerfile командой docker compose build: сборка заняла 1 мин 33 с, размер итогового образа — 767 МБ. Сокращение достигнуто за счёт базового образа python:3.11-slim вместо python:3.11 и вынесения компиляторов в отдельную стадию builder.

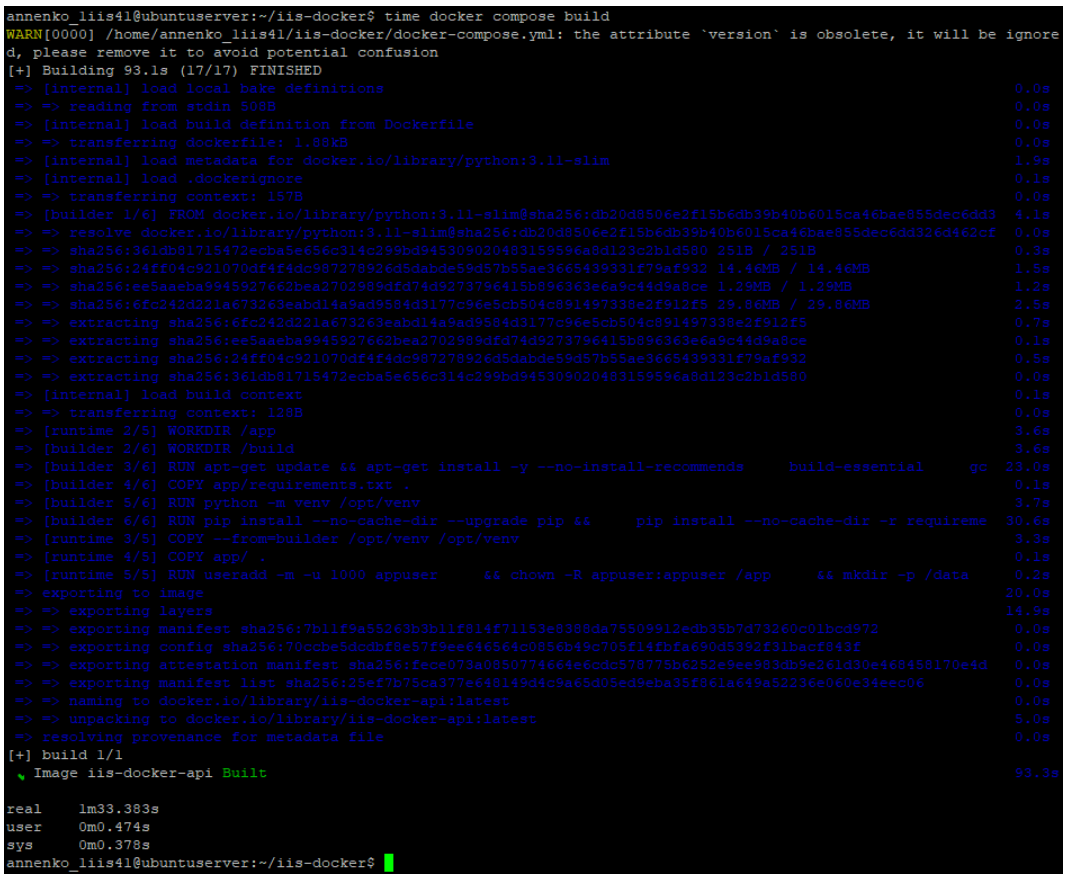


Рисунок 10 – Сборка образа командой docker compose build (1 мин 33 с)

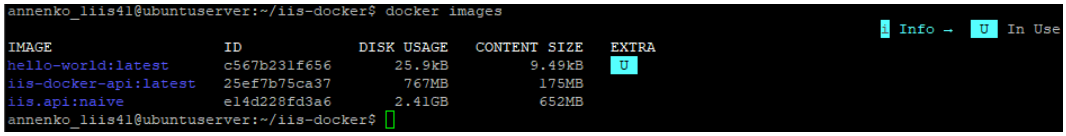


Рисунок 11 – Размеры образов после оптимизации (767 МБ)

Результаты сравнения приведены в таблице 1.

Таблица 1 – Сравнение размеров образов

Образ	Dockerfile	Размер на диске	Сжатый размер
iis-api:naive	одна стадия, python:3.11	2,41 ГБ	652 МБ
iis-docker-api	multi-stage, python:3.11-slim	767 МБ	175 МБ

Размер образа уменьшился на 68 % ((2,41 − 0,767) / 2,41).

Шаг 7. Запуск системы

Командой docker compose up -d созданы сеть iis-docker_default, тома model_data и postgres_data и запущены контейнеры iis-docker-api-1 и iis-docker-db-1. Команда docker compose ps показывает, что оба сервиса находятся в состоянии Up, порты 8000 и 5432 опубликованы на хосте.

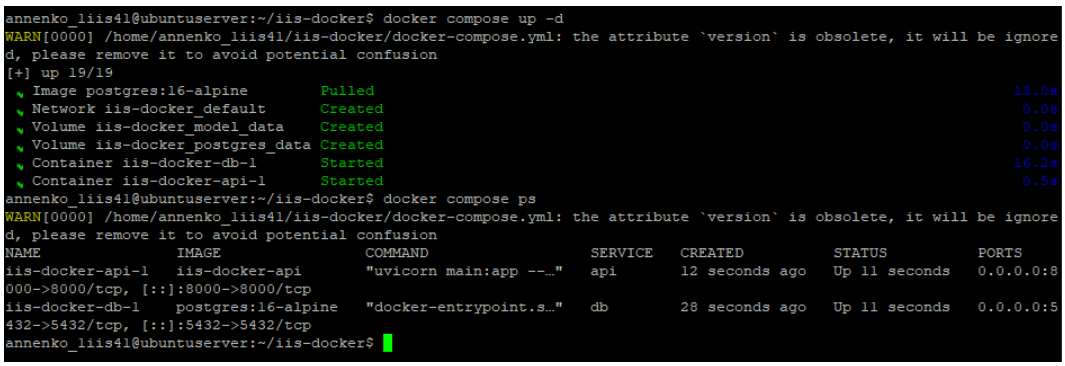


Рисунок 12 – Запуск системы и состояние сервисов (docker compose ps)

Шаг 8. Проверка работоспособности API

Корневой эндпоинт GET / вернул {"status":"ok","service":"IIS Iris Classifier"}. Эндпоинт POST /predict принимает четыре признака цветка и возвращает предсказанный класс: для трёх наборов признаков получены классы 0, 1 и 2.

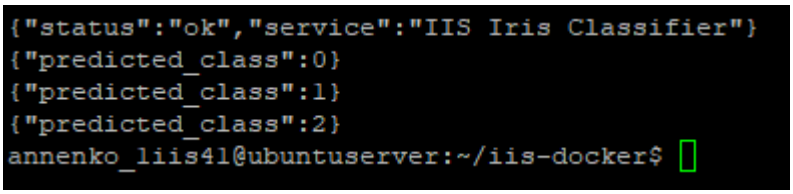


Рисунок 13 – Ответы API на запросы curl (классы 0, 1 и 2)

Дополнительно проверил работу через веб-интерфейс Swagger по адресу http://192.168.0.5:8000/docs: запрос POST /predict выполнен кнопкой Execute, сервер вернул код 200 и {"predicted_class": 0}.

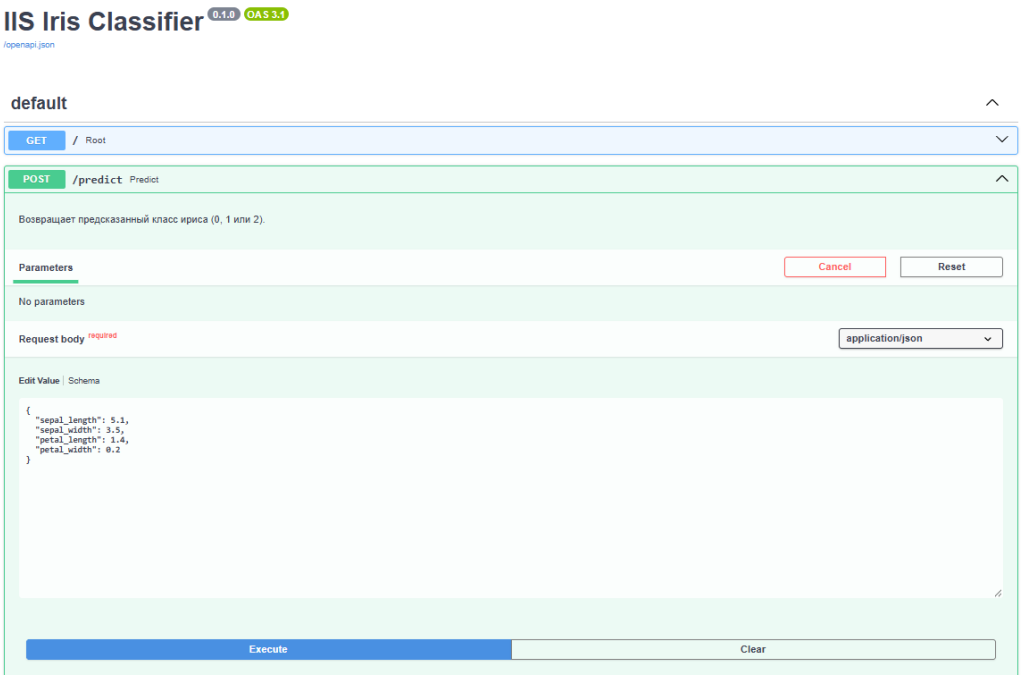


Рисунок 14 – Выполнение запроса POST /predict в Swagger UI

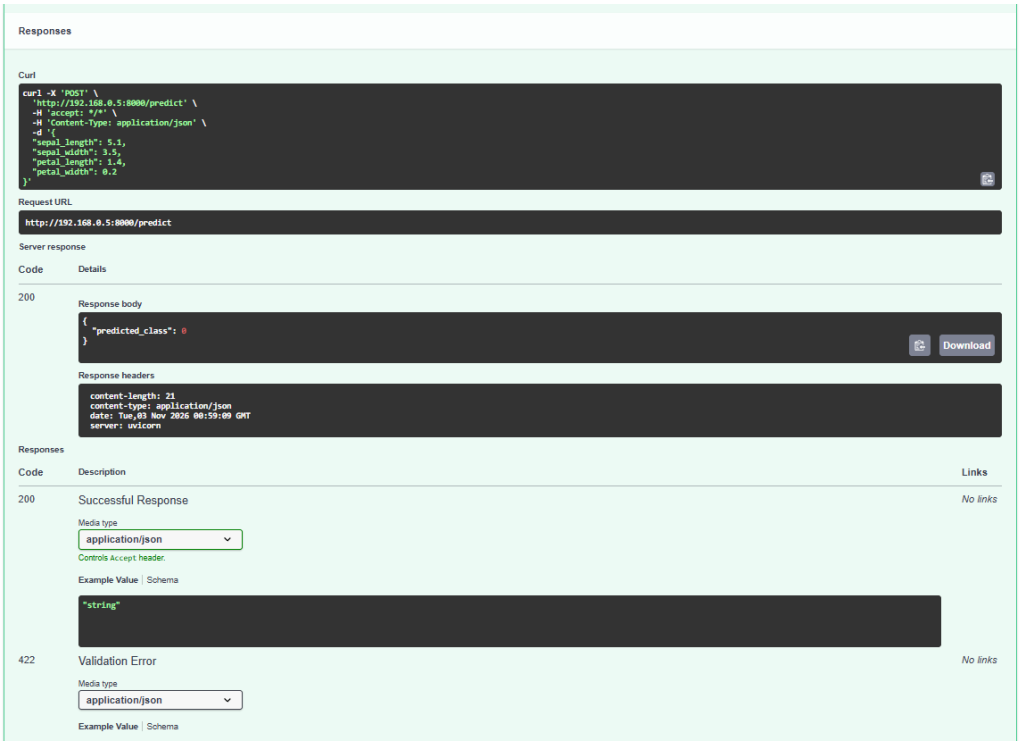


Рисунок 15 – Ответ сервера: код 200, predicted_class = 0

Шаг 9. Проверка тома model_data

Команда docker volume ls показывает тома iis-docker_model_data и iis-docker_postgres_data. Команда docker volume inspect iis-docker_model_data выводит точку монтирования /var/lib/docker/volumes/iis-docker_model_data/_data, то есть данные хранятся вне контейнера.

```
annenko_iis41@ubuntuserver:~/iis-docker$ docker volume ls
DRIVER    VOLUME NAME
local     iis-docker_model_data
local     iis-docker_postgres_data
annenko_iis41@ubuntuserver:~/iis-docker$ docker volume inspect iis-docker_model_data
[
    {
        "CreatedAt": "2026-11-03T00:47:19Z",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "0454132b5de796fbd924938ccccedlfa93b449401c8ca6bcf5371blfe898c209",
            "com.docker.compose.project": "iis-docker",
            "com.docker.compose.version": "5.7.0",
            "com.docker.compose.volume": "model_data"
        },
        "Mountpoint": "/var/lib/docker/volumes/iis-docker_model_data/_data",
        "Name": "iis-docker_model_data",
        "Options": null,
        "Scope": "local"
    }
]
annenko_iis41@ubuntuserver:~/iis-docker$
```

Рисунок 16 – Список томов и параметры тома model_data

Внутри контейнера в каталоге /data находится файл model.joblib размером 186 753 байта, владелец — appuser. Значит модель была обучена при первом запуске и сохранена в том.

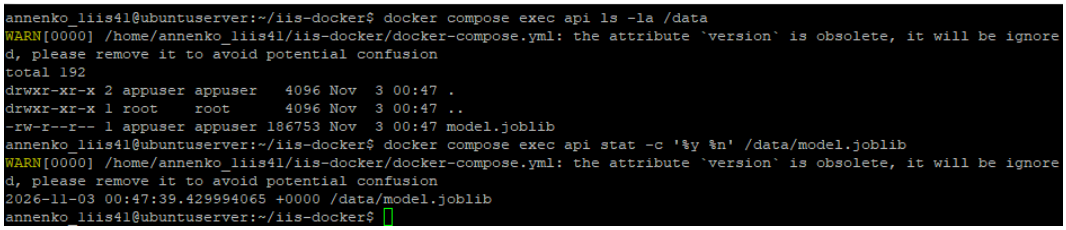


Рисунок 17 – Содержимое каталога /data в контейнере api

Шаг 10. Проверка сохранности данных при пересоздании контейнеров

Я зафиксировал время изменения файла модели (2026-11-03 00:47:39), затем выполнил docker compose down — контейнеры api и db были удалены, и команда docker compose ps не показала ни одного контейнера. После docker compose up -d контейнеры созданы заново, а время изменения файла /data/model.joblib осталось прежним.

Это подтверждает, что модель хранится в именованном томе и не переобучается при пересоздании контейнеров, а данные не теряются при остановке и удалении контейнеров.

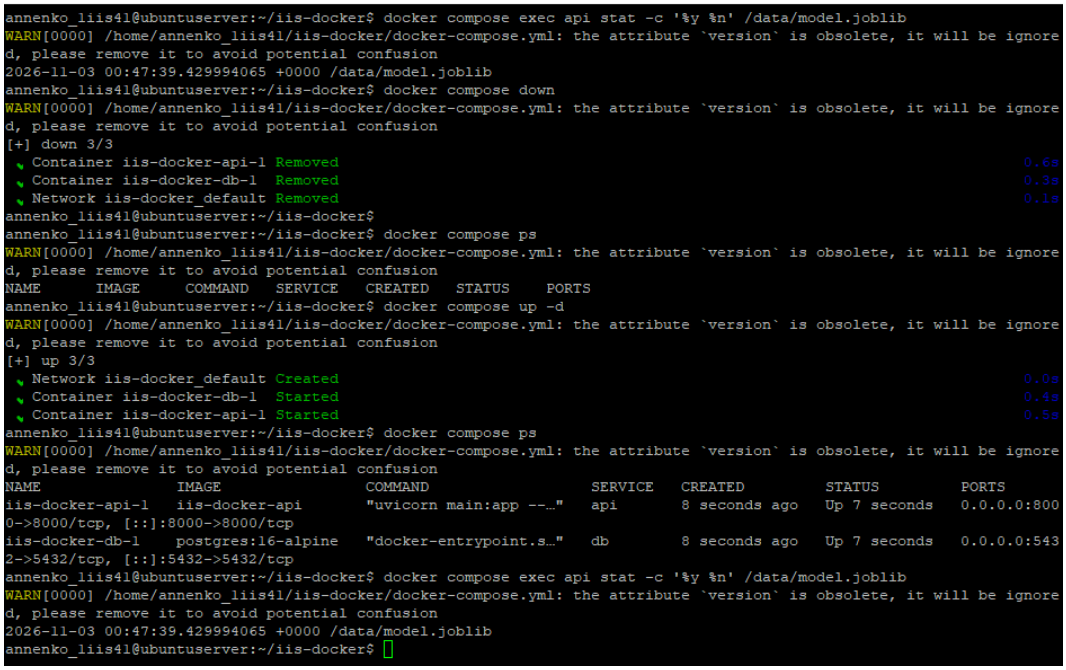


Рисунок 18 – Проверка сохранности модели после docker compose down и up -d

Результаты проверки приведены в таблице 2.

Таблица 2 – Сохранность данных при пересоздании контейнеров

Параметр	До перезапуска	После перезапуска
Время изменения /data/model.joblib	2026-11-03 00:47:39	2026-11-03 00:47:39
Состояние контейнеров (docker compose ps)	api, db — Up	api, db — Up

Вывод

В ходе лабораторной работы я освоил контейнеризацию приложения машинного обучения средствами Docker: научился писать multi-stage Dockerfile, описывать многоконтейнерное приложение в docker-compose.yml и организовывать персистентное хранение данных через именованные тома.

Размер образа уменьшен с 2,41 ГБ до 767 МБ, то есть на 68 %, за счёт использования базового образа python:3.11-slim и вынесения компиляторов в стадию builder; время сборки оптимизированного образа составило 1 мин 33 с.

Проверено, что API отвечает на запросы (классы 0, 1 и 2), модель сохраняется в томе model_data и не переобучается после docker compose down и повторного docker compose up -d, а приложение работает от непривилегированного пользователя.